

Network and Computer Security — Lecture 12

Dr. Karim Lounis

Jan - Jun 2026



TLS 1.3 Phases (with more details)

TLS 1.3: Step 1 — Client Hello

The Client sends the list of supported cipher suites and TLS version, a random $\#$, session ID, and many extensions: server name (here www.cia.gov) and its key share (by default an uncompressed X25519 point) — [pcap file available](#).

```

  Transport Layer Security
  TLSv1.3 Record Layer: Handshake Protocol: Client Hello
    Content Type: Handshake (22)
    Version: TLS 1.0 (0x0301)
    Length: 558
  Handshake Protocol: Client Hello
    Handshake Type: Client Hello (1)
    Length: 554
    Version: TLS 1.2 (0x0303)
    Random: 6da1fe348da51b4ed9cebfdc84881e7c3a6dfa26ba025873db691d9df68f591
    Session ID Length: 32
    Session ID: 8e4c27e4cdb31389b6b6301f7c9043ba7081d8198e8fb718ee9059e4b0eb22c9
    Cipher Suites Length: 32
    Cipher Suites (16 suites)
    Compression Methods Length: 1
    Compression Methods (1 method)
    Extensions Length: 449
    Extension: Reserved (GREASE) (len=0)
    Extension: compress_certificate (len=3)
    Extension: psk_key_exchange_modes (len=2)
    Extension: status_request (len=5)
    Extension: signed_certificate_timestamp (len=0)
    Extension: key_share (len=43) x25519
      Type: key_share (51)
      Length: 43
    Key Share extension
      Client Key Share Length: 41
      Key Share Entry: Group: Reserved (GREASE), Key Exchange length: 1
      Key Share Entry: Group: x25519, Key Exchange length: 32
        Group: x25519 (29)
        Key Exchange Length: 32
        Key Exchange: 03e3a6148f1b7b7401459e730041081e99cee1ebd1486ff4110ae553f421234e
      Extension: renegotiation_info (len=1)
      Extension: encrypted_client_hello (len=250)
      Extension: supported_versions (len=7) TLS 1.3, TLS 1.2
      Extension: ec_point_formats (len=2)
      Extension: session_ticket (len=0)
      Extension: server_name (len=16) name=www.cia.gov
```

TLS 1.3: Step 2 — Server Hello, Change Cipher Spec, Server Finished, and Encrypted Application Data

The Server responds with a Server Hello message containing a random $\#$, session ID, the selected cipher suite (e.g., here TLS_AES_128_GCM_SHA256), and the Server's key share (an uncompressed X25519 point).

```

  Transport Layer Security
  TLSv1.3 Record Layer: Handshake Protocol: Server Hello
    Content Type: Handshake (22)
    Version: TLS 1.2 (0x0303)
    Length: 122
  Handshake Protocol: Server Hello
    Handshake Type: Server Hello (2)
    Length: 118
    Version: TLS 1.2 (0x0303)
    Random: 1ce9c6bbd40b2cea00419fa3adc87acbaa0c96a9585be292db3fe237f04fd144
    Session ID Length: 32
    Session ID: 6368ce210b63b00a23babe2d63a2f028c07f8086258ab941c58f9a87a0223f16
    Cipher Suite: TLS_AES_128_GCM_SHA256 (0x1301)
    Compression Method: null (0)
    Extensions Length: 46
  Extension: supported_versions (len=2) TLS 1.3
    Type: supported_versions (43)
    Length: 2
    Supported Version: TLS 1.3 (0x0304)
  Extension: key_share (len=36) x25519
    Type: key_share (51)
    Length: 36
  Key Share extension
    Key Share Entry: Group: x25519, Key Exchange length: 32
    [JA3S Fullstring: 771,4865,43-51]
    [JA3S: f4fbc55ea12b31ae17cfb7e614afda8]
  TLSv1.3 Record Layer: Change Cipher Spec Protocol: Change Cipher Spec
    Content Type: Change Cipher Spec (20)
    Version: TLS 1.2 (0x0303)
    Length: 1
    Change Cipher Spec Message
  TLSv1.3 Record Layer: Application Data Protocol: Hypertext Transfer Protocol
    Opaque Type: Application Data (23)
    Version: TLS 1.2 (0x0303)
    Length: 42
  Encrypted Application Data: d26f5afba2b44cba658d089c4dae6f797a61f1be2734051617baee6d04ad4398da4af58576bf05db943de
  [Application Data Protocol: Hypertext Transfer Protocol]
```

TLS 1.3: Step 2 — Server Hello, Change Cipher Spec, Server Finished, and Encrypted Application Data

The messages that follow the “Server Hello” message are **encrypted** using the derived key (from both key shares). Those **encrypted messages** contain:

Source	Destination	Protocol	Length	Info
192.168.193.1...	www.ensia.edu.dz	TCP	78	58344 → https(443) [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=36952049 TSecr=0 SACK...
www.ensia.edu...	192.168.193.170	TCP	76	https(443) → 58344 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1400 SACK_PERM TSval=2324125...
192.168.193.1...	www.ensia.edu.dz	TCP	66	58344 → https(443) [ACK] Seq=1 Ack=1 Win=131840 Len=0 TSval=36952091 TSecr=2324125881
192.168.193.1...	www.ensia.edu.dz	TLSv1.3	306	Client Hello (SNI=www.ensia.edu.dz)
www.ensia.edu...	192.168.193.170	TCP	68	https(443) → 58344 [ACK] Seq=1 Ack=241 Win=73728 Len=0 TSval=2324125933 TSecr=36952092
www.ensia.edu...	192.168.193.170	TLSv1.3	1466	Server Hello, Change Cipher Spec
www.ensia.edu...	192.168.193.170	TCP	1466	https(443) → 58344 [ACK] Seq=1401 Ack=241 Win=73728 Len=1400 TSval=2324125937 TSecr=369520...
192.168.193.1...	www.ensia.edu.dz	TCP	66	58344 → https(443) [ACK] Seq=241 Ack=2801 Win=129024 Len=0 TSval=36952134 TSecr=2324125937
www.ensia.edu...	192.168.193.170	TLSv1.3	96	Application Data
192.168.193.1...	www.ensia.edu.dz	TCP	66	58344 → https(443) [ACK] Seq=241 Ack=2829 Win=131008 Len=0 TSval=36952146 TSecr=2324125937
192.168.193.1...	www.ensia.edu.dz	TLSv1.3	146	Change Cipher Spec, Application Data
www.ensia.edu...	192.168.193.170	TCP	68	https(443) → 58344 [ACK] Seq=2829 Ack=321 Win=73728 Len=0 TSval=2324126024 TSecr=36952152
192.168.193.1...	www.ensia.edu.dz	TLSv1.3	89	Application Data
www.ensia.edu...	192.168.193.170	TCP	68	https(443) → 58344 [ACK] Seq=2829 Ack=344 Win=73728 Len=0 TSval=2324128519 TSecr=36954678

- Server’s certificate chain (Certificate Message).
- Encrypted extensions message (e.g., SNI, ALPN, etc).
- CertificateVerify (prove certificate ownership + bind to handshake).
- Finished message (prove correct handshake).

All these messages **are** shown as “Application Data”

TLS 1.3: Step 2 — Server Hello, Change Cipher Spec, Server Finished, and Encrypted Application Data

The messages that follow the “Server Hello” message are **encrypted** using the derived key (from both key shares). Those **encrypted messages** contain:

Source	Destination	Protocol	Length	Info
192.168.193.1	www.ensia.edu.dz	TCP	78	58344 → https(443) [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=36952049 TSecr=0 SACK_PERM
www.ensia.edu.dz	192.168.193.170	TCP	76	https(443) → 58344 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1400 SACK_PERM TSval=2324125881
192.168.193.1	www.ensia.edu.dz	TCP	66	58344 → https(443) [ACK] Seq=1 Ack=1 Win=131840 Len=0 TSval=36952091 TSecr=2324125881
192.168.193.1	www.ensia.edu.dz	TLSv1.3	306	Client Hello (SNI=www.ensia.edu.dz)
www.ensia.edu.dz	192.168.193.170	TCP	68	https(443) → 58344 [ACK] Seq=1 Ack=241 Win=73728 Len=0 TSval=2324125933 TSecr=36952092
www.ensia.edu.dz	192.168.193.170	TLSv1.3	1466	Server Hello, Change Cipher Spec
www.ensia.edu.dz	192.168.193.170	TCP	1466	https(443) → 58344 [ACK] Seq=1401 Ack=241 Win=73728 Len=1400 TSval=2324125937 TSecr=36952092
192.168.193.1	www.ensia.edu.dz	TCP	66	58344 → https(443) [ACK] Seq=241 Ack=2801 Win=129024 Len=0 TSval=36952134 TSecr=2324125937
www.ensia.edu.dz	192.168.193.170	TLSv1.3	96	Encrypted Extensions, Certificate, Certificate Verify, Finished
192.168.193.1	www.ensia.edu.dz	TCP	66	58344 → https(443) [ACK] Seq=241 Ack=2829 Win=131008 Len=0 TSval=36952146 TSecr=2324125937
192.168.193.1	www.ensia.edu.dz	TLSv1.3	146	Change Cipher Spec, Finished
www.ensia.edu.dz	192.168.193.170	TCP	68	https(443) → 58344 [ACK] Seq=2829 Ack=321 Win=73728 Len=0 TSval=2324126024 TSecr=36952152
192.168.193.1	www.ensia.edu.dz	TLSv1.3	89	Application Data
www.ensia.edu.dz	192.168.193.170	TCP	68	https(443) → 58344 [ACK] Seq=2829 Ack=344 Win=73728 Len=0 TSval=2324128519 TSecr=36954678

- Server’s certificate chain (Certificate Message).
- Encrypted extensions message (e.g., SNI, ALPN, etc).
- CertificateVerify (prove certificate ownership + bind to handshake).
- Finished message (prove correct handshake).

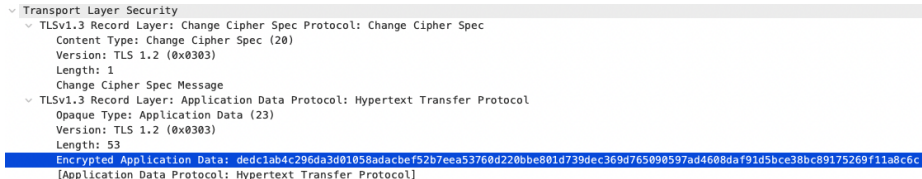
All these messages **were** shown as “Application Data”,

TLS 1.3: Step 3 — Change Cipher Spec, Client Finished, and Encrypted Application Data

The Client checks the certificate shared by the Server, generates symmetric keys as it has the key share of the Server, and sends the “Change Cipher Spec” and “Client Finished” messages.

The server uses the “CertificateVerify” message to sign the hash of the handshake and prove ownership of the public key declared in the certificate.

The “Finished” messages allow both parties to prove to each other the correct derivation of the session key (using the HMAC of the ciphersuite).



From this point, both the Client and the Server start communicating by encrypting messages.

TLS Phases — TLS 1.3

This illustrates TLS handshake with the ENSIA website (It uses TLSv1.3 — 05/2024):

No.	Time	Source	Destination	Protocol	Le Info
6.11.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TCP	→ 58314 → https(443) [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=64 TSval=2853412732 TSecr=0 SACK_PERM
6.17.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TCP	→ https(443) → 58314 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1460 SACK_PERM TSval=1835749019 TSecr=2853412732 WS=128
6.17.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TCP	→ 58314 → https(443) [ACK] Seq=1 Ack=1 Win=131712 Len=0 TSval=2853412789 TSecr=1835749019
6.17.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TLSv1.3	→ Client Hello (SNI=www.ensia.edu.dz)
6.26.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TCP	→ https(443) → 58314 [ACK] Seq=1 Ack=518 Win=30080 Len=0 TSval=1835749071 TSecr=2853412789
6.26.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TLSv1.3	→ Server Hello, Change Cipher Spec, Application Data
6.26.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TCP	→ 58314 → https(443) [ACK] Seq=518 Ack=1349 Win=130368 Len=0 TSval=2853412885 TSecr=1835749072
6.26.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TCP	→ https(443) → 58314 [ACK] Seq=1349 Ack=518 Win=30080 Len=1348 TSval=1835749072 TSecr=2853412789 [TCP segment of a reas...
6.26.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TCP	→ 58314 → https(443) [ACK] Seq=518 Ack=2697 Win=129024 Len=0 TSval=2853412885 TSecr=1835749072
6.26.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TLSv1.3	→ Application Data, Application Data, Application Data
6.26.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TCP	→ 58314 → https(443) [ACK] Seq=518 Ack=3679 Win=130048 Len=0 TSval=2853412886 TSecr=1835749072
6.29.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TLSv1.3	→ Change Cipher Spec, Application Data
6.29.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TLSv1.3	→ Application Data
6.35.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TLSv1.3	→ Application Data
6.35.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TLSv1.3	→ Application Data
6.35.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TCP	→ 58314 → https(443) [ACK] Seq=1049 Ack=3966 Win=130752 Len=0 TSval=2853412969 TSecr=1835749195
6.35.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TCP	→ 58314 → https(443) [ACK] Seq=1049 Ack=4253 Win=130496 Len=0 TSval=2853412969 TSecr=1835749195
6.39.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TCP	→ https(443) → 58314 [ACK] Seq=4253 Ack=1049 Win=31104 Len=0 TSval=1835749241 TSecr=2853412914
7.58.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TCP	→ https(443) → 58314 [ACK] Seq=4253 Ack=1049 Win=31104 Len=1348 TSval=1835750434 TSecr=2853412969 [TCP segment of a rea...
7.58.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TCP	→ 58314 → https(443) [ACK] Seq=1049 Ack=5601 Win=129664 Len=0 TSval=2853414206 TSecr=1835750434
7.59.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TCP	→ https(443) → 58314 [ACK] Seq=5601 Ack=1049 Win=31104 Len=1348 TSval=1835750434 TSecr=2853412969 [TCP segment of a rea...
7.59.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TCP	→ 58314 → https(443) [ACK] Seq=1049 Ack=6949 Win=129664 Len=0 TSval=2853414208 TSecr=1835750434
7.59.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TCP	→ https(443) → 58314 [ACK] Seq=6949 Ack=1049 Win=31104 Len=1348 TSval=1835750434 TSecr=2853412969 [TCP segment of a rea...
7.59.	192.168.193.172	ensia.edu.dz	ensia.edu.dz	TCP	→ 58314 → https(443) [ACK] Seq=1049 Ack=8707 Win=130656 Len=0 TSval=2853414210 TSecr=1835750434

In RFC8446: “All handshake messages after the ServerHello are now encrypted. The newly introduced **Encrypted Extensions message** allows various extensions previously sent in the clear in the ServerHello (e.g., SNI, ALPN, etc.) to also enjoy confidentiality protection”

TLS 1.2 and 1.3 Local Decryption using OpenSSL

Demo

```
openssl s_client -connect mdn.dz:443 -tls1_3 -keylogfile ./file.keys
```

TLS features

The following should be considered about TLS:

- TLS 1.3 use ephemeral key exchange algorithms (e.g., ECDHE) that provide Perfect Forward Secrecy (PFS). This means even if an attacker compromises the server's private key, they cannot decrypt past AND future sessions protected by the TLS handshake.
- According to IETF (Internet Engineering Task Force), TLS 1.2 average time to complete the handshake process is 300ms. Whereas in TLS 1.3 it's been reduced to 200ms.
- There can be variations in the handshake depending on the specific TLS version and chosen cipher suite. Some ciphers might not require server authentication.
- It differs drastically from SSL/TLS prior to TLS 1.3, where key exchange happened AFTER authentication. In SSL and TLS 1-1.2 certificates were sent as plaintext.

IPSec: Internet Protocol Security

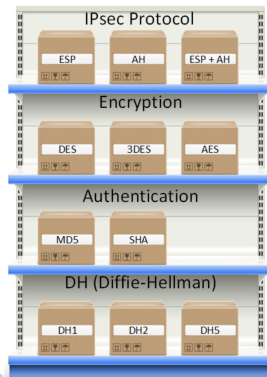
Internet Protocol Security

Definition

IPsec. It is a set of protocols (suite) that cooperate to secure IP communications (IPv4 and IPv6) between two hosts in an untrusted network. The protocols operate at the network layer, securing all what sits above.

It has the following features:

- IPsec is used to setup VPNs (Virtual Private Networks) — encrypted tunnel btw two hosts.
- It allows the protection of lower-layer protocols, e.g., ICMP, IGMP, RIP, etc.
- Provides mutual authentication, message integrity, confidentiality, and non-repudiation.
- The IETF formed the IP Security Working Group in 1992 to standardize IPsec.
- IPsec main components are: AH, ESP, and SAs.
- IPsec SAs are generated following the ISAKMP framework, implemented as IKEv1 and IKEv2.



Internet Protocol Security

Definition

IPsec main components:

- **AH (Authentication Header)**. Ensure the integrity of IP packets (nobody can tamper-with IP packets).
- **ESP (Encapsulating Security Payload)**. Ensure the confidentiality of IP packets (nobody can read the IP data).
- **SA (Security Association)**. Define how traffic should be secured (cipher suite, lifetime, IDs, etc).
- **ISAKMP (Internet Security Association and Key Management protocol)**. A framework that defines how to negotiate SAs and the structure of the key exchange messages.
- **IKE (Internet Key Exchange)**. Implement the specifications of ISAKMP. There are two versions, IKEv1 and IKEv2. They run over UDP 500 (isakmp) and UDP 4500 (nat-t).

Internet Protocol Security

Definition

IPsec. It is a set of protocols (suite) that cooperate to secure IP communications (IPv4 and IPv6) between two hosts in an untrusted network. The protocols operate at the network layer, securing all what sits above.

It aims at setting up encrypted tunnels (IPSec Tunnels) between peers accross the Internet (it is a layer 3 end-to-end security scheme).

IPsec aims to stop the following attacks:

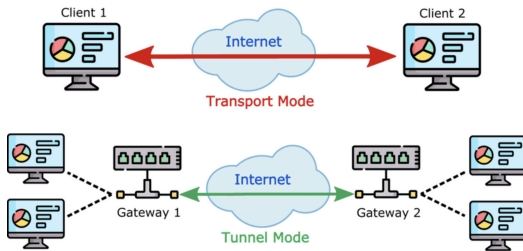
- IP datagram eavesdropping: By encrypting Layer 4 and above.
- IP datagram modification: By enfocing message authentication.
- IP datagram spoofing: By enforcing entirly authentication.
- IP datagram replay: By using data freshness.
- MITM attacks: By combining encryption and authentication.
- DoS attacks (Partially): Thanks to anti-spoofing and cookies.
- Session hijacking: By combining encryption and authentication.

Internet Protocol Security

Definition

IPsec. It is a set of protocols (suite) that cooperate to secure IP communications (IPv4 and IPv6) between two hosts in an untrusted network. The protocols operate at the network layer, securing all what sits above.

It aims at setting up encrypted tunnels (IPSec Tunnels) between peers across the Internet (it is a layer 3 end-to-end security scheme):



IPsec supports two operational modes: transport and tunnel.

Internet Protocol Security

Definition

IPSec

must

Th

It a

acc

Sup

IPsec supports two operational modes:

- **Transport Mode:**

Here, the protection (by ESP) concerns the payload of the IP packet. The header is neither modified nor encrypted. If AH is used, the IP header is authenticated and replay-protected, but sent in plaintext (i.e., we can know who talks to who).

- ☞ Has issues with NAT. Requires NAT-T (NAT traversal).
- ☞ Some Layer 3 attacks are still possible.

- **Tunnel Mode:**

Here, the entire IP packet is protected (encrypted and authenticated). Packets are encapsulated into new IP packet with new headers. This mode is used to create VPNs.

- ☞ Net-to-net, host-to-net, and host-to-host VPNs.
- ☞ No issues with NAT.

com-
work.
ove.

bers

Internet Protocol Security

Definition

Security Association. It is a unidirectional digital contract (a data structure) between two parties that stores the set of security parameters, algorithms, and keys that will be used latter on to establish a secure communication using IPSec protocol, i.e., AH and ESP.

Two types: (1) IKE SA (a.k.a., Parent SA), and (2) IPsec SA (a.k.a., Child SAs). The former is established to securely establish the latter.

The structure consists of:

- ① Security Parameter Index — SPI: Determine which SA per packet. It is a 32-bit value for IPsec SAs and 128-bit value for IKE SAs.
- ② A sequence number used to prevent replay attacks. Start at 0 and increments for each packet with an anti-replay window (e.g., 2^{32}).
- ③ IP protocols in use: ESP for encryption and authentication, and AH for authentication only.

Internet Protocol Security

Definition

Security Association. It is a unidirectional digital contract (a data structure) between two parties that stores the set of security parameters, algorithms, and keys that will be used latter on to establish a secure communication using IPSec protocol, i.e., AH and ESP.

The structure consists of:

- ④ Encryption algorithms for ESP (e.g., AES, 3DES, ChaCha20).
- ⑤ Integrity/Hashing algorithms (e.g., HMAC-SHA-384).
- ⑥ Encryption and message authentication keys.
- ⑦ Mode of operation: Transport or Tunnel.
- ⑧ Livetime: Determines when SA expires and must be rekeyed.
- ⑨ Traffic selectors: Define which traffic is protected (IP address range, port range, protocol, etc).

ISAKMP Phases (IKEv1) ([pcap file](#))

IPSec Phases (IKEv1)

IPSec (IKEv1) consists of two main phases:

① Phase 1 (Main mode):

- Consists of exchanging 6 ISAKMP messages to establish an IKE SA.
- **M1.** The initiator sends a 64-bit initiator security parameter index (iSPI), **nonce**, proposed crypt-algo (encryption, integrity, and authentication method — E.g., Pre-Shared Key, certificate, etc).
- **M2.** The responder replies with a 64-bit responder security parameter index (rSPI), **nonce**, selected crypt-algo (encryption, integrity, and authentication method) from the proposed ones.
- In IKEv1 the exchanged SPIs are called cookies.
- **M3.** Initiator sends his DH public value along with a nonce.
- **M4.** Responder sends his DH public value along with a nonce too.
- At this stage, both parties derives a DH key.
- Using the PRF agreed-upon, the exchanged nonces, and the derived DH key, both compute a set of three keys (sk_e , sk_a , and sk_d).

IPSec Phases (IKEv1) — Con't

IPSec (IKEv1) consists of two main phases:

① Phase 1 (Main mode):

- **M5.** The initiator sends his identity IDi (e.g., username, certificate, or FQ domain name) and an authentication proof (i.e., hash the derived key with PSK, cookie, and IDi). M5 is encrypted using sk_e .
- **M6.** The responder sends a his identity IDr (e.g., username, certificate, or FQ domain name) and an authentication proof (i.e., hash derived key with PSK, cookie. and IDr). M6 is encrypted using sk_e .
- The IKE_SA is created, and can be used to secure the next step.

② Phase 1 (Quick mode): Establishes an IPsec SA:

- Exchanging 3 ISAKMP messages to establish an IKE SA.
- **M7.** The initiator sends proposed crypt-algo, nonce, [DH value], traffic selectors, and an integrity code (protected using sk_e and sk_a).
- **M8.** The responder sends chosen crypt-algo, nonce, [DH value], traffic selectors, and an integrity code (protected using sk_e and sk_a).
- **M9.** The initiator confirms with a hash on previously exchanged messages (i.e., M7 and M8).

IPSec Phases (IKEv1) — Con't

IPSec consists of two main phases:

Source	Destination	Protocol	Length	Info
192.168.12.1	192.168.12.2	ISAKMP	210	Identity Protection (Main Mode)
192.168.12.2	192.168.12.1	ISAKMP	150	Identity Protection (Main Mode)
192.168.12.1	192.168.12.2	ISAKMP	326	Identity Protection (Main Mode)
192.168.12.2	192.168.12.1	ISAKMP	346	Identity Protection (Main Mode)
192.168.12.1	192.168.12.2	ISAKMP	150	Identity Protection (Main Mode)
192.168.12.2	192.168.12.1	ISAKMP	118	Identity Protection (Main Mode)
192.168.12.1	192.168.12.2	ISAKMP	262	Quick Mode
192.168.12.2	192.168.12.1	ISAKMP	262	Quick Mode
192.168.12.1	192.168.12.2	ISAKMP	102	Quick Mode

This is a Wireshark pcap capture for an IPsec connection where two parties run IKEv1 to protect IP packets (starting from **Message 5**).

- > User Datagram Protocol, Src Port: isakmp (500), Dst Port: isakmp (500)
- ✓ Internet Security Association and Key Management Protocol
 - Initiator SPI: e47a591fd057587f
 - Responder SPI: a00b8ef0902bb8ec
 - Next payload: Identification (5)
 - > Version: 1.0
 - Exchange type: Identity Protection (Main Mode) (2)
 - > Flags: 0x01
 - Message ID: 0x00000000
 - Length: 108
 - Encrypted Data (80 bytes)

IPSec Phases (IKEv1) — Con't

Demo (IKEv1)

ISAKMP Phases (IKEv2) ([pcap file](#))

IPSec Tunnel mode Phases (IKEv2)

IPSec tunnel mode consists of two main phases:

① IKE Phase 1 — INIT:

- The peers aim to establish an IKE SA using two ISAKMP messages.
- The initiator (using `IKE_SA_INIT` req) sends a 64-bit initiator security parameter index (iSPI), nonce, proposed crypt-algo (encryption, integrity, PRF, and DH groups), along with [DH public value].
- The responder (using `IKE_SA_INIT` response) replies with a 64-bit responder security parameter index (rSPI), nonce, selected crypt-algo (encryption, integrity, PRF, and DH group) from the proposed ones, along with [DH public value]. SPIs are combined into 128-bit SPI.
- At this stage, both parties derive a DH key.
- Using the PRF agree-upon, the exchanged nonces, and the derived DH key, both compute a set of three keys (sk_e , sk_a , and sk_d).
- The `IKE_SA` is created, and can be used to secure the next step.

IPSec Phases (IKEv2)

IPSec tunnel mode consists of two main phases:

② IKE Phase 2 — AUTH:

- The peers aim to establish an IPsec SA using two secure ISAKMP message (secure due to IKE SA from previous phase).
- The initiator (using IKE_AUTH req) sends an identifier (e.g., username, certificate, FQ domain name), [a certificate], a certification request, a signature over the IKE_INIT messages or hash with PSK (to prove identity), agreed-upon crypt-algo, and traffic selectors.
- The initiator (using IKE_AUTH resp) sends an identifier (e.g., username, certificate, FQ domain name), [a certificate], a signature over the IKE_INIT message or hash with PSK (to prove identity), agreed crypt-algo (encryption, integrity, ESP or AH), and traffic selectors.
- Both parties use s_d from IKE SA and the exchanged nonces to derive encryption and message authentication keys for the IPsec SA.
- The IPsec SA is made of two unidirectional SAs (one for each traffic direction), each with its own SPI (Security Parameter Index) and independent cryptographic parameters (algorithms & keys).

IPSec Phases (IKEv2) — Con't

IPSec tunnel mode consists of two main phases:

Source	Destination	Protocol	Length	Info
192.168.245.1...	172.16.15.92	ISAKMP	346	IKE_SA_INIT MID=00 Initiator Request
172.16.15.92	192.168.245.131	ISAKMP	351	IKE_SA_INIT MID=00 Responder Response
192.168.245.1...	172.16.15.92	ISAKMP	1326	IKE_AUTH MID=01 Initiator Request
172.16.15.92	192.168.245.131	ISAKMP	1318	IKE_AUTH MID=01 Responder Response
192.168.245.1...	172.16.15.92	ESP	162	ESP (SPI=0xac0faf03)
172.16.15.92	192.168.245.131	ESP	162	ESP (SPI=0xc1a9656b)
192.168.245.1...	172.16.15.92	ESP	162	ESP (SPI=0xac0faf03)
172.16.15.92	192.168.245.131	ESP	162	ESP (SPI=0xc1a9656b)
192.168.245.1...	172.16.15.92	ESP	162	ESP (SPI=0xac0faf03)
172.16.15.92	192.168.245.131	ESP	162	ESP (SPI=0xc1a9656b)
192.168.245.1...	172.16.15.92	ESP	162	ESP (SPI=0xac0faf03)
172.16.15.92	192.168.245.131	ESP	162	ESP (SPI=0xc1a9656b)

This is a Wireshark pcap capture for an IPSec connection where two parties agree to use ESP to protect their IP packets.

Note that there are two SPIs per traffic bound. One SPI for inbound traffic (e.g., 0xac0faf03) and another one for outbound (e.g., 0xc1a9656b).

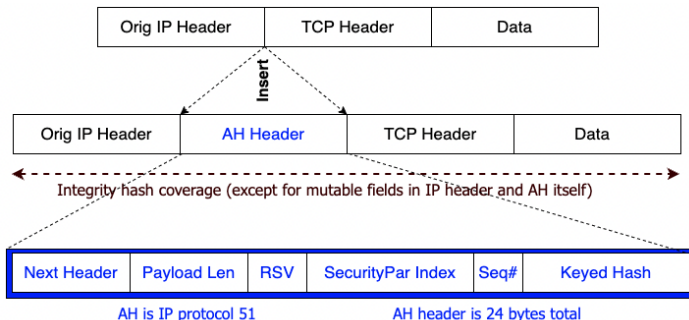
IPSec Phases (IKEv2) — Con't

Demo (IKEv2)

IPSec: Protection with AH and ESP

Authentication Header (AH)

It guarantees data origin authentication, data freshness, and data integrity. It does not ensure data confidentiality.

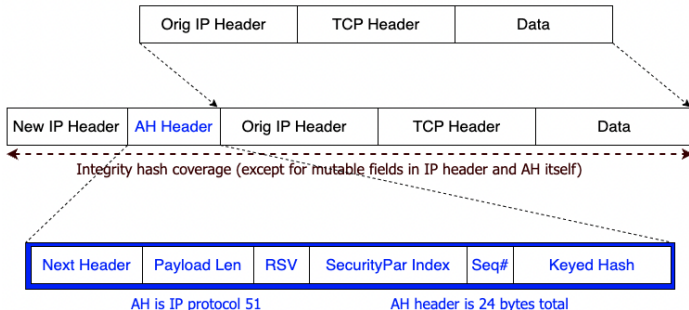


AH header in transport mode

IPSec: Protection with AH and ESP

Authentication Header (AH)

It guarantees data origin authentication, data freshness, and data integrity. It does not ensure data confidentiality.



AH header in tunnel mode

IPSec: Protection with AH and ESP

Authentication Header (AH)

It guarantees data origin, authentication, data freshness, and data integrity. The AH header's fields are interpreted as follows:

- Next Header (8 bits): upper-layer protocol (tcp:6, udp: 17, etc).
- Payload length (8 bits): in 4-byte unites. Size of the header itself.
- Reserved (16 bits): For future use.
- Security parameters index (32 bits): SPI (tells which SA to use here).
- Sequence number (32 bits): Prevents replay attacks.
- Integrity check value (multiple of 32 bits in IPv4): HMAC value.

AH is IP protocol 51

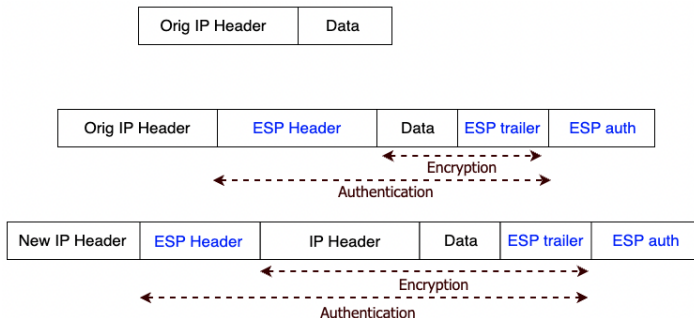
AH header is 24 bytes total

AH header in tunnel mode

IPSec: Protection with AH and ESP

Encapsulating Security Payload (ESP)

It provides origin authenticity through source authentication, data integrity through hash functions and confidentiality through encryption protection for IP packets.



ESP header in both modes.

IPSec: Protection with AH and ESP

Encapsulating Security Payload (ESP)

It provides origin authenticity through source authentication, data integrity. The interpretation of ESP information fields is as follows:

- P
- ESP header: consists of:
 - Security parameter index (32 bits) — (which SA to use here).
 - Sequence number (32 bits) for packets freshness.
 - ESP trailer:
 - Padding (0-255 bytes).
 - Padding length (8 bits).
 - Next header (8 bits).
 - ESP Auth: Integrity check value, a multiple of 32 bits.

Authentication

ESP header in both modes.

IPSec: Protection with AH and ESP

Typical IP headers are not authenticated. Hence, they are subject to IP spoofing. We use **IPsec-AH** to solve this issue:

- AH can be used to authenticate the packets. Authentication is applied to the entire packet, but the mutable fields are set to 0.

Typical IP headers are not encrypted. Hence, they can be intercepted and disclosed. We use **IPsec-ESP** to solve this issue:

- ESP can be used to encrypt the packets.
- AH does authenticate the outer IP header in tunnel mode, whereas ESP doesn't.
- Sequence numbers are never reused, because a new security association must be renegotiated before an attempt to increment the sequence number beyond its maximum value.

VPN doesn't mean IPsec

Not all VPNs use IPsec. There are several other VPN technologies, each with different use cases. E.g., TLS-based VPNs (OpenVPN, AnyConnect, etc) provide encryption of application-layer payload only.

Configuring VPN between two Linux Machine

You can use **StrongSwan** (open-source IPsec VPN solution for Unix-like OSs) to set up an IPsec VPN tunnel between two local machines:

```
scooper@kali: ~  
File Actions Edit View Help  
GNU nano 7.2 /etc/ipsec.conf *  
ipsec.conf - strongSwan IPsec configur  
# basic configuration  
config setup  
    charondebug="all"  
    uniqueids=yes  
# Sample VPN connections  
conn %default  
    ikelifetime=60m  
    keylife=20m  
    rekeymargin=3m  
    keyingtries=1  
    keyexchange=ikev2  
    authby=secret  
conn my-vpn  
    left=192.168.193.220  
    leftsubnet=192.168.193.0/24  
    right=192.168.193.131  
    rightsubnet=192.168.193.0/24  
    authby=secret  
    ike=aes256-sha2-256-modp1024!  
    esp=aes256-sha2-256!  
    auto=start
```

```
scooper@kali: ~  
File Actions Edit View Help  
GNU nano 7.2 /etc/ipsec.secrets *  
# This file holds shared secrets or RSA private key  
# RSA private key for this host, authenticating it  
# which knows the public part.  
192.168.193.220 192.168.193.131 : PSK "lounie2025"
```

IKEv1 (RFC 2409) Vs IKEv2 (RFC 7296)

IKEv2 is more efficient, secure, and modern than IKEv1, with fewer exchanges, mandatory PFS, and better support for mobility and rekeying:

- IKEv1 and IKEv2 are not interoperable.
- IKEv1 uses 9 messages, whereas IKEv2 uses 3.
- IKEv2 has DoS protection built-in (using cookie).
- IKEv2 has anti-replay protection mandatory.
- IKEv1 re-negotiates SA during Phase 2.
- IKEv2 has perfect forward secrecy (new DH for each child SA).
- IKEv1-PSK (aggressive mode) is weaker than IKEv2-PSK.
- IKEv2 natively supports ECDH groups.
- IKEv2 supports mobility (MOBIKE).

IKEv1 (RFC 2409) was first used in 1998. The IKEv2 was published in 2005, and finalized in 2014 (RFC 7296) — **and still people use IKEv1**.

- **End.**